

High-Radix/Mixed-Radix NTT Multiplication Algorithm/Architecture Co-Design Over Fermat Modulus

Yile Xing , *Student Member, IEEE*, Guangyan Li , Zewen Ye , *Student Member, IEEE*,
Ryan W. L. Luk , *Member, IEEE*, Donglong Chen , *Member, IEEE*, Hong Yan , *Life Fellow, IEEE*,
and Ray C. C. Cheung , *Senior Member, IEEE*

Abstract—Polynomial multiplication using Number Theoretic Transform (NTT) is crucial in lattice-based post-quantum cryptography (PQC) and fully homomorphic encryption (FHE), with modulus q significantly affecting performance. Fermat moduli of the form $2^{2^n} + 1$, such as 65537, offer efficiency gains due to simplified modular reduction and powers-of-2 twiddle factors in NTT. While Fermat moduli have been directly applied or explored for incorporation into existing schemes, Fermat NTT-based polynomial multiplication designs remain underexplored in fully exploiting the benefits of Fermat moduli. This work presents a high-radix/mixed-radix NTT architecture tailored for Fermat moduli, which improves the utilization of the powers-of-2 twiddle factors in large transform sizes. In most cases, our design achieves a 30%–85% reduction in DSP area-time product (ATP) and a 70%–100% reduction in BRAM ATP compared to state-of-the-art designs with smaller or equivalent modulus, while maintaining competitive LUT and FF ATP, underscoring the potential of Fermat NTT-based polynomial multipliers in lattice-based cryptography.

Index Terms—Polynomial multiplication, Fermat number transform, high radix, mixed radix, conflict-free memory access.

I. INTRODUCTION

LATTICE-BASED cryptography offers strong security guarantees and efficient arithmetic for both post-quantum cryptography (PQC) and fully homomorphic encryption (FHE). A key operation in lattice-based schemes is polynomial multiplication based on the number theoretic transform (NTT), which plays an important role in overall efficiency [1], [2], [3].

The use of Fermat moduli in NTT offers significant advantages [4], [5], [6]. Fermat moduli take the form of $F_n = 2^b + 1 = 2^{2^n} + 1$, enabling efficient modular reduction via $x \bmod F_n = (x_{low} - x_{high}) \bmod F_n$, where $x_{low} = x \bmod 2^b$ and $x_{high} = \lfloor x/2^b \rfloor$. Moreover, its modular multiplications by powers of 2 can be done with shifts alone [7]. Fermat number transform (FNT), which refers to an NTT over a Fermat modulus, enables the use of power-of-2 twiddle factors, thus reducing computational complexity. However, the power-of-two-based twiddle factors impose a strict constraint $N \leq 2^{n+1}$ on the transform size N , which also corresponds to the polynomial degree. This limitation makes it challenging to directly support many existing parameter sets, as most PQC and FHE schemes rely on non-Fermat moduli and large N .

Currently, a notable scheme that does not face the challenges associated with Fermat moduli is Hawk [8], which explicitly recommends the Fermat modulus $F_4 = 65537$ in its specification document. Another modulus option in Hawk is 12289. Hawk is the only lattice-based candidate in the second round of the Additional Digital Signatures for the NIST PQC standardization process.

Additionally, although the Dilithium specification does not recommend a Fermat modulus, Abdulrahman et al. [6] propose switching the original large modulus 8380417 to a small Fermat modulus 257 for computing the small products cs_1 and cs_2 in Dilithium. The switching results in a speed-up of 33.1%–37.6% for the relevant operations (basemul + INTT) of the signing procedure on Cortex-M4.

Most importantly, Kim et al. [4] recently address the challenge by proposing a strategy to integrate Fermat moduli into the FHE scheme CKKS, which could potentially be extended to other similar scenarios involving non-Fermat moduli and very large transform sizes. The strategy consists of two key steps:

Received 7 October 2024; revised 10 June 2025; accepted 7 July 2025. Date of publication 21 July 2025; date of current version 10 September 2025. This work was supported in part by Hong Kong Innovation and Technology Commission (InnoHK Project CIMDA), in part by the CityUHK Project under Grant 9440356, in part by ITF Project under Grant ITS/098/22, in part by Guangdong Provincial Key Laboratory of IRADS under Grant 2022B1212010006, in part by Guangdong and Hong Kong Universities “1+1+1” Joint Research Collaboration Scheme, in part by Guangdong Basic and Applied Basic Research Foundation under Grant 2024A1515011274, in part by Guangdong Province General Universities Key Field Project (New Generation Information Technology) under Grant 2023ZDZX1033, and in part by UIC Research under Grant UICR04202401-21. Recommended for acceptance by K. Gaj. (Corresponding author: Donglong Chen.)

Yile Xing, Guangyan Li, Ryan W. L. Luk, Hong Yan, and Ray C. C. Cheung are with the Department of Electrical Engineering, City University of Hong Kong, Hong Kong SAR 999077, China (e-mail: ylxing2-c@my.cityu.edu.hk; guangyanli5-c@my.cityu.edu.hk; ryanluk5-c@my.cityu.edu.hk; h.yan@cityu.edu.hk; r.cheung@cityu.edu.hk).

Zewen Ye is with the College of Information Science & Electronic Engineering, Zhejiang University, Hangzhou 310027, China, and also with the Department of Electrical Engineering, City University of Hong Kong, Hong Kong SAR 999077, China (e-mail: lucas.zw.ye@outlook.com).

Donglong Chen is with Beijing Normal-Hong Kong Baptist University, Zhuhai 519087, China (e-mail: donglongchen@uic.edu.cn).

Digital Object Identifier 10.1109/TC.2025.3590972

1). Adoption of auxiliary Fermat modulus - After decomposing the original a very large modulus Q into small components q_i using the RNS technique, a sufficiently large auxiliary Fermat modulus is introduced to replace q_i . 2). Polynomial Ring Transformation - The original large polynomial degree N is decomposed into smaller sub-transforms N_i , enabling efficient computations using Fermat moduli.

Developing efficient FNT-based polynomial multiplication in hardware is essential not only for supporting existing schemes like [8] and methods such as [6], but also for facilitating the transition of approach in [4] to other potential schemes. Furthermore, it encourages more research focused on Fermat modulus-based cryptographic schemes, pursuing breakthroughs in high efficiency from a parameter perspective.

However, FNT-based polynomial multiplication design remains insufficiently explored. One challenge is how to better leverage FNT's property of powers-of-2 twiddle factors.

Regarding the transform size constraint $N \leq 2^{n+1}$ for enabling power-of-2 twiddle factors, an optical communication work [9], based solely on simulations of a fully unrolled structure, demonstrates that high-radix structures can effectively utilize power-of-2 twiddle factors even when $N > 2^{n+1}$. Moreover, high-radix NTT enables greater parallelism and throughput while reducing the number of computation cycles and memory accesses, regardless of the chosen modulus. To leverage these advantages, we design our FNT-based polynomial multiplier using high-radix and mixed-radix architectures, where the mixed-radix design specifically addresses cases where N is not a power of the radix R .

As high-radix designs introduce highly parallel processing, they may result in excessive resource consumption and significant frequency degradation when radix R is large. Most existing high-radix/mixed-radix designs, primarily based on R2 and R4 configurations over other moduli, have not adequately addressed these issues. In the context of Fermat moduli, the challenges can be framed as follows: How can multiplications by powers-of-2 and those involving the unavoidable non-powers-of-2 twiddle factors be efficiently handled? Additionally, how can resource reuse mechanisms be further optimized? Our work goes beyond merely adopting Fermat moduli, it tackles these challenges and fully exploits the properties of FNT in ways that prior works have not explored.

A. Related Works

1) *NTT Over Fermat Modulus*: Ma et al. [5] propose an FPGA implementation of PQC schemes with fixed parameters $q = 65537$ and $N = 256$. This work adopts a simple R2-NTT architecture with a special modular multiplier design. Combined with D1 representation, the partial products are obtained by shifting and using multiplexers, followed by an accumulation process to construct the modular multiplier without using DSP. This work does not leverage the property that twiddle factors in FNT can be powers of 2. Instead, it simply takes advantage of the fact that, under the Fermat modulus, multiplication by powers of 2 avoids modular reduction operation.

Works in other fields [9], [10], [11], [12], [13], [14], [15] aim to utilize FNT to achieve a high-accuracy convolution at a lower complexity compared to FFT-based methods in their areas. Although these works have different applications, the techniques combined with FNT are versatile. One important technique is the D1 representation, which is usually combined with FNT to simplify modular multiplication by a power of 2.

2) *High-Radix/Mixed-Radix NTT Hardware Design*: In cryptographic hardware works, high-radix NTT over general moduli has been explored in works [16], [17], [18], [19], [20]. However, the radix R is subject to two limitations in these works. First, R is typically limited to 4. Due to the complicated NTT/INTT butterfly operations, more datapaths and interconnect networks [16], [20], higher radix will dramatically increase the resource usage. Second, R is usually constrained to be a factor of N , limiting the configuration flexibility.

Duong-Ngoc and Lee [21] propose an R4&R2 mixed-radix NTT architecture which can break the second limitation. However, the design does not reuse R2 and R4 butterfly units in each stage, leading to substantial resource usage.

Li et al. [22], Guo and Li [23] explore the split-radix NTT, which has a lower theoretical complexity than R4 NTT. However, the asymmetry of the split-radix butterfly complicates the implementation, and the algorithm cannot fully exploit its theoretical advantages in pipelined hardware implementations.

Many works explore designs with multiple butterfly units in parallel [16], [17], [24], [25], [26]. Generally, by applying the $m \times \text{radix-}R$ ($m, R \geq 2$) butterfly style instead of 1*R2 style, the ideal cycles is reduced from $\mathcal{O}(\frac{N}{2} \log N)$ to $\mathcal{O}(\frac{N}{Rm} \log_R N)$. However, it has been shown that an NTT kernel with multiple R2 butterfly units is less efficient than an R4 NTT kernel [16].

Therefore, a unified mixed-radix architecture that can support large radix- R as well as more N is a promising research direction to improve efficiency.

As for the conflict-free memory mapping for the temporary data, Zhang et al. [24] propose a 2*R2 design and supports 4 data in parallel but require 8 address generators to determine the reading and writing addresses. Chen et al. [16] reduce the number of address generators by half and insert registers to get writing addresses from reading addresses.

B. Major Contributions

This paper proposes a polynomial multiplier based on high-radix/mixed-radix FNT. The main contributions are as follows:

- We propose high-radix and mixed-radix NTT/INTT with merged pre-/post-processing algorithms. Specifically, the transform processes are modularized, ensuring that D1 representation is employed for concentrated multiplications by power-of-2 constants in R2NTT/R2INTT module, while retaining the benefits of normal representation for multiplications by normal numbers in ModMuls module. This modularization also facilitates PWM calculations and supports complete polynomial multiplication.
- We introduce a low-complexity conflict-free memory mapping scheme and extend it to support mixed-radix operations. This method reduces the complexity of memory

mapping and improves frequency and efficiency in high-radix cases by requiring only one bank index as the select signal for memory mapping interconnections.

- We propose a polynomial multiplication architecture based on FNT, configurable to a broader range of degree N and radix R configurations. The architecture accommodates radix $R = 4, 8, 16$ and their mixed combinations, providing flexibility to meet varying latency and area constraints with high efficiency. The architecture allows small-radix stages to reuse the R2NTT, R2INTT and ModMul modules from large-radix stages.
- Our implementation shows that the R8 and R16 configurations offer comparable or better efficiency than the R4 configuration. In most cases, our design achieves a 30%–85% reduction in DSP ATP and a 70%–100% reduction in BRAM ATP compared to state-of-the-art designs with smaller or equivalent modulus, while maintaining competitive LUT and FF ATP.

II. PRELIMINARY

A. NTT-Based Polynomial Multiplication

NTT is generally a Discrete Fourier Transform (DFT) over a finite field $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$. NTT and its inverse transform (INTT) are defined as:

$$A_k = \text{NTT}_N(a)_k = \sum_{n=0}^{N-1} a_n \omega_N^{nk} \bmod q, \quad k = 0, 1, \dots, N-1$$

$$a_n = \text{INTT}_N(A)_n = N^{-1} \sum_{k=0}^{N-1} A_k \omega_N^{-nk} \bmod q,$$

$$n = 0, \dots, N-1$$

where ω_N is a primitive N -th root of unit in $\mathbb{Z}_q$, i.e., $\omega_N^N \equiv 1 \pmod{q}$ and $\omega_N^i \not\equiv 1 \pmod{q}$ for any $i = 1, \dots, N-1$. NTT also has the circular convolution property analogous to DFT. Similar to the Fast Fourier transform (FFT) for DFT, NTT supports a divide-and-conquer method to reduce the computational complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(N \log N)$. An iterative R2 DIT-NTT_{NR} algorithm and DIF-INTT_{RN} algorithm are provided in Algorithms 1 and 2 respectively.

In lattice-based PQC schemes, the polynomial is mainly defined over the ring $\mathbb{Z}_q[x]/\langle x^N + 1 \rangle$. Suppose two polynomials $a(x) = \sum_{n=0}^{N-1} a_n x^n$ and $b(x) = \sum_{n=0}^{N-1} b_n x^n$, the polynomial multiplication can be performed with the negative wrapped convolution method based on NTT. The coefficients of the product $a(x)b(x) = \sum_{n=0}^{N-1} c_n x^n$ can be expressed by:

$$c_n = (\text{INTT}_N(\text{NTT}_N(a_n \cdot \omega_{2N}^n) \odot \text{NTT}_N(b_n \cdot \omega_{2N}^n)))_n \cdot \omega_{2N}^{-n}$$

where ω_{2N} is a primitive $(2N)$ _{th} root of unit in $\mathbb{Z}_q$, thereby $\omega_N \equiv \omega_{2N}^2 \pmod{q}$. The point-wise multiplication inside NTTs and outside INTT are called pre-processing and post-processing respectively.

Researchers make efforts to simplify the negative weighted convolution by improving the FFT-like structure in NTT/INTT. The works successfully eliminates the pre-processing [27], the post-processing [28], the scaling factor N^{-1} [24] and also the

Algorithm 1: R2NTT: Radix-2 DIT-NTT_{NR}

Input : A vector a of length N , modulus q , N -th primitive root of unity ω_N in $\mathbb{Z}_q$.

Output: Output vector A .

```

1 for  $s \leftarrow 0$  to  $\log(N) - 1$  do
2    $M \leftarrow 2^{s+1}$ ;
3    $\omega_M \leftarrow \omega_N^{N/M} \bmod q$ ;
4    $\Delta_{\text{idx}} \leftarrow N/2^{s+1}$ ;
5   for  $b \leftarrow 0$  to  $2^s - 1$  do
6      $\omega \leftarrow \omega_M^{\text{bit-rev}(b)} \bmod q$ ;
7     for  $g \leftarrow 0$  to  $N/2^{s+1} - 1$  do
8        $\text{idx} \leftarrow b \cdot N/2^s + g$ ;
9        $T \leftarrow \omega \cdot a_{\text{idx} + \Delta_{\text{idx}}} \bmod q$ ;
10       $a_{\text{idx} + \Delta_{\text{idx}}} \leftarrow a_{\text{idx}} - T \bmod q$ ;
11       $a_{\text{idx}} \leftarrow a_{\text{idx}} + T \bmod q$ ;
12  $A \leftarrow a$ ;
```

Algorithm 2: R2INTT: Radix-2 DIF-INTT_{RN}

Input : A vector A of length N , modulus q , inversion of a N -th primitive root of unity ω_N^{-1} in $\mathbb{Z}_q$.

Output: Output vector a .

```

1 for  $s \leftarrow \log(N) - 1$  to 0 do
2    $M \leftarrow 2^{s+1}$ ;
3    $\omega_M^{-1} \leftarrow (\omega_N^{-1})^{N/M} \bmod q$ ;
4    $\Delta_{\text{idx}} \leftarrow N/2^{s+1}$ ;
5   for  $b \leftarrow 0$  to  $2^s - 1$  do
6      $\omega^{-1} \leftarrow (\omega_M^{-1})^{\text{bit-rev}(b)} \bmod q$ ;
7     for  $g \leftarrow 0$  to  $N/2^{s+1} - 1$  do
8        $\text{idx} \leftarrow b \cdot N/2^s + g$ ;
9        $T \leftarrow 2^{-1}(A_{\text{idx}} + A_{\text{idx} + \Delta_{\text{idx}}}) \bmod q$ ;
10       $A_{\text{idx} + \Delta_{\text{idx}}} \leftarrow 2^{-1}\omega^{-1}(A_{\text{idx}} - A_{\text{idx} + \Delta_{\text{idx}}}) \bmod q$ ;
11       $A_{\text{idx}} \leftarrow T$ ;
12  $a \leftarrow A$ ;
```

bit-reverse operation before NTT and after INTT [16], which contribute to a scramble-free FFT-like structure for NTT/INTT with $\omega_{2N}^{\pm n}$ and N^{-1} merged in each stage.

B. NTT Over Fermat Modulus and Diminished-1 Representation

NTT over Fermat number modulus, i.e., $q = F_n = 2^b + 1 = 2^{2^n} + 1$, is known as the Fermat number transform (FNT) [29], which enables shifting operations to replace multiplications inside transforms when applying the power-of-2 twiddle factors. This property highly depends on the relationship between q and N . Table I lists two commonly used modulus F_3 and F_4 , their corresponding root of unity ω_N and the degree N . For commonly used Fermat modulus F_2 to F_6 , the twiddle factors are all a power of 2 when $N \leq 2^{n+1}$ [29].

TABLE I
THE ROOT OF UNITY ω_N FOR DIFFERENT N IN FNT

F_n	ω_2	ω_4	ω_8	ω_{16}	ω_{32}	ω_{64}	ω_{128}	ω_{256}	...	ω_{65536}
$F_3 = 257$	2^8	2^4	2^2	2	$2^{\frac{1}{2}}$	$2^{\frac{1}{4}}$	$2^{\frac{1}{8}}$	$2^{\frac{1}{16}}$		
$F_4 = 65537$	2^{16}	2^8	2^4	2^2	2	$2^{\frac{1}{2}}$	$2^{\frac{1}{4}}$	$2^{\frac{1}{8}}$	...	$2^{\frac{1}{4096}}$

Algorithm 3: Arithmetics Operations in Diminished-1

```

Norm_to_D1(in)      Addition(in1, in2)
if in == 0 then      if in1 ==  $2^b$  then
  | out  $\leftarrow 2^b$ ;    | out  $\leftarrow$  in2;
else                else if in2 ==  $2^b$  then
  | out  $\leftarrow$  in - 1; | out  $\leftarrow$  in1;
return out          else
                      | out  $\leftarrow$  in1 + in2;
                      | out  $\leftarrow$  out[b - 1 : 0] + ~out[b];
D1_to_Norm(in)      return out
if in ==  $2^b$  then  -----
  | out  $\leftarrow$  0;      Negation(in)
else                if in ==  $2^b$  then
  | out  $\leftarrow$  in + 1; | out  $\leftarrow 2^b$ ;
return out          else
                      | out  $\leftarrow \{1'b0, \sim \text{in}[b - 1 : 0]\}$ ;
Mul_by_2k(k, in)   return out
if in ==  $2^b$  then  -----
  | out  $\leftarrow$  in;      Mul_by_2k(k, in)
else                if in ==  $2^b$  then
  | if k ≥ 0 then        | out  $\leftarrow$  in;
  | | out  $\leftarrow$  |k|-bit left invert circular shift for in[b - 1 : 0]
  | else
  | | out  $\leftarrow$  |k|-bit right invert circular shift for in[b - 1 : 0]
return out

```

Considering $b + 1$ bits is required for representing all the possible results for Fermat number $F_n = 2^b + 1$, our work does not use only b bits to represent data or ignore the number 2^b as [29]. It is because we would like to avoid a highly improbable but plausible chance of the polynomial coefficients being affected by errors when the value -1 occurs during NTT computation. Instead, we follow [7] to apply D1 representation, which represents the original number x as $x - 1$. b bits are used to represent numbers from 1 to 2^b , while an extra bit is used to represent 0. For example, with modulus $F_2 = 17 = 10001B$, the number from $00001B$ to $10000B$ in normal binary representation is represented as $00000B$ to $01111B$ in D1 respectively, while the number $00000B$ is represented as $10000B$.

The computations about D1 are listed in Algorithm 3. The bit-width of inputs and outputs are $b + 1$. The utilization of D1 reduces modular multiplication by constant powers of 2 to approximately a circular shift.

C. Memory Access Conflicts

Scalability across varying numbers of parallel butterfly units and different radices is essential for managing parallelism

and optimizing the throughput rate in polynomial multiplication. Two main memory access strategies exist for NTT architectures: constant-geometry NTT and in-place NTT. In constant geometry NTT, as adopted by [31], [32], butterfly **inputs and outputs are stored separate memory banks, with a ping-pong buffering scheme managing data flow**. This structure inherently avoids read-the-same-bank (RSB) conflicts but still requires mechanisms to handle read-after-write (RAW) conflicts. To eliminate redundant storage, some works, such as [16], [17], [24], [30], [33], favor in-place NTT, where temporary data are read from and written to the same addresses within the banks. This approach requires addressing both RSB and RAW conflicts, which we also tackle in our design.

RSB conflict occurs when different data points in the same bank are required in the same cycle. [30], [33] propose a similar compact memory scheme to avoid the RSB conflict. Assuming there are R banks for the temporary data, the mapped bank index and address for each data point with an original address are as follows:

$$\text{OrigAddr} = b_{\log N-1} \dots \underbrace{b_{2 \log R-1} \dots b_{\log R}}_{\log R \text{ bits}} \underbrace{b_{\log R-1} \dots b_0}_{\log R \text{ bits}},$$

$$\text{BankIndex} = \sum_{i=0}^{\lceil \frac{\log N}{\log R} \rceil - 1} \text{OrigAddr}[(\log R) \cdot (i+1) : (\log R)i] \bmod R,$$

$$\text{BankAddr} = b_{\log N-1} \dots b_{2 \log R-1} \dots b_{\log R}.$$

A 4-bank memory mapping is shown in Fig. 1(a). Zhang et al. [24] note that this method still encounters an RSB conflict in one stage of a 2^*R2 design, as illustrated in Fig. 1(b), and they handle this stage specifically. This design supports 4 operations in parallel but requires 8 address generators to determine the bank indexes for reading and writing. Chen et al. [16] and Mu et al. [17] address this by new schemes with increased complexity. Chen et al. [16] reduce the number of address generators by half and inserts registers to get writing addresses from reading addresses. Fortunately, the original scheme introduces no RSB conflict when there is only one radix- R butterfly unit. Our design further reduces the address generator to only 1, minimizing logic and register usage.

RAW conflict occurs when the data is fetched again before it completes the pipeline of the previous stage, as shown in Fig. 1(c). The cycles between reading and storage should be less or equal to the minimum cycles between fetching the same data points, i.e., $\frac{N}{mR^2}$, in an $m^* \text{radix-}R$ design.

This type of conflict is usually not a concern in most works where N is large or R is small. However, it is very likely to happen in high-radix designs. Mu et al. [17] pursue a fully-pipelined design, sacrificing many choices of R when RAW conflict happens. Our design simply inserts stalls to avoid RAW conflict as Fig. 1(d). The added cycles and the registers consumption due to stalls are acceptable compared to the gaining from high radix, especially when N is relatively large.

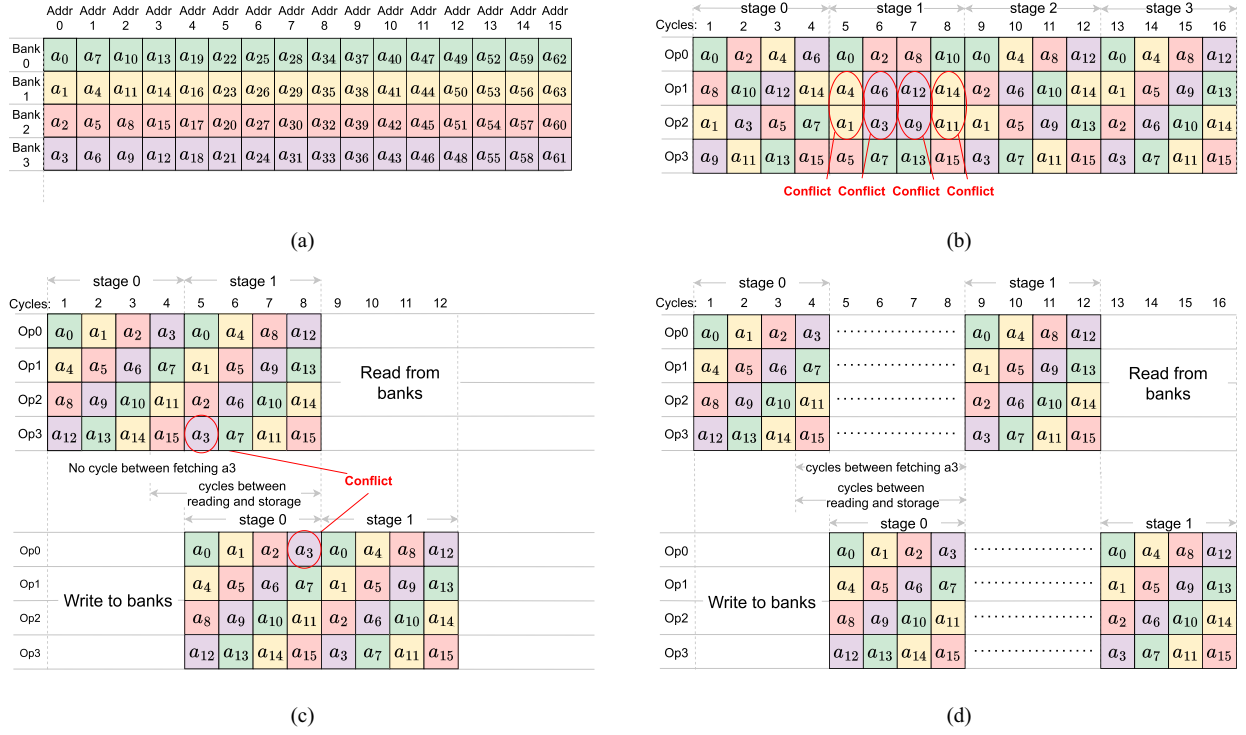


Fig. 1. Two types of conflict. The bank indexes are indicated by different colors. (a): 4-bank memory mapping by [30] (≤ 64 points). (b): A read-the-same-bank (RSB) conflict example in 16-points 2*R2 design. $\{a_4, a_1\}$, $\{a_6, a_3\}$, $\{a_{12}, a_9\}$ and $\{a_{14}, a_{11}\}$ are required at the same cycle but located in the same bank. (c): A read-after-write (RAW) conflict example in 16-point 1*R4 design. a_3 is read again in stage 2 before its result from stage 0 is written into the bank. (d): Inserting stalls to delay the writing operations thereby avoiding RAW conflict.

III. HIGH-RADIX/MIXED-RADIX NTT AND INTT

Building upon the derivation ideas from [16], [34], we provide a detailed derivation of high-radix DIT-NTT and DIF-INTT with pre/post-processing. Furthermore, we extend the discussion by proposing our high-radix/mixed-radix algorithms specifically tailored for FNT and its inverse transform.

A. High-Radix DIT-NTT With Merged Pre-Processing

The radix- R length- N DIT-NTT with merged pre-processing can be obtained by

$$\begin{aligned} \bar{A}_k &= \sum_{n=0}^{N-1} a_n \omega_{2N}^n \omega_N^{kn} \bmod q \\ &= \sum_{i=0}^{\frac{N}{R}-1} a_{Ri} \omega_{2N}^{(2k+1)(Ri)} + \sum_{i=0}^{\frac{N}{R}-1} a_{Ri+1} \omega_{2N}^{(2k+1)(Ri+1)} \\ &\quad + \dots + \sum_{i=0}^{\frac{N}{R}-1} a_{Ri+R-1} \omega_{2N}^{(2k+1)(Ri+R-1)} \bmod q \\ &= \sum_{i=0}^{\frac{N}{R}-1} a_{Ri} \omega_{2N}^{(2k+1)(Ri)} + \omega_{2N}^{(2k+1)} \sum_{i=0}^{\frac{N}{R}-1} a_{Ri+1} \omega_{2N}^{(2k+1)(Ri)} \\ &\quad + \dots + \omega_{2N}^{(R-1)(2k+1)} \sum_{i=0}^{\frac{N}{R}-1} a_{Ri+R-1} \omega_{2N}^{(2k+1)(Ri)} \bmod q. \end{aligned}$$

By denoting $\{a_{Ri+r}\}$ as $\{s_{i,r}\}$, the above equation can be written as

$$\begin{aligned} \bar{A}_k &= \sum_{i=0}^{\frac{N}{R}-1} s_{i,0} \omega_{2N}^{(2k+1)i} + \omega_{2N}^{(2k+1)} \sum_{i=0}^{\frac{N}{R}-1} s_{i,1} \omega_{2N}^{(2k+1)i} \\ &\quad + \dots + \omega_{2N}^{(R-1)(2k+1)} \sum_{i=0}^{\frac{N}{R}-1} s_{i,R-1} \omega_{2N}^{(2k+1)i} \bmod q \\ &= \sum_{r=0}^{R-1} \omega_{2N}^{r(2k+1)} \sum_{i=0}^{\frac{N}{R}-1} s_{i,r} \omega_{2N}^{(2k+1)i} \bmod q, \end{aligned}$$

where $k = 0, \dots, N-1$. Due to the periodicity, it can be further expressed as

$$\begin{aligned} \bar{A}_{l+\frac{N}{R}j} &= \sum_{r=0}^{R-1} \omega_{2N}^{r(2(l+\frac{N}{R}j)+1)} \sum_{i=0}^{\frac{N}{R}-1} s_{i,r} \omega_{2N}^{(2(l+\frac{N}{R}j)+1)i} \bmod q \\ &= \sum_{r=0}^{R-1} \omega_{2N}^{rj} \omega_{2N}^{r(2l+1)} \sum_{i=0}^{\frac{N}{R}-1} s_{i,r} \omega_{2N}^{(2l+1)i} \bmod q, \end{aligned}$$

where $l = 0, \dots, \frac{N}{R}-1, j = 0, \dots, R-1$. The steps of NTT with merged pre-processing can be summarized as:

- 1) Derive R $\frac{N}{R}$ -point NTT with merged pre-processing $\{\bar{S}_{l,r} | l = 0, \dots, \frac{N}{R}-1\}$ ($r = 0, \dots, R-1$) of $\{s_{i,r} | i = 0, \dots, \frac{N}{R}-1\}$ ($r = 0, \dots, R-1$) from

$\{a_n | n = 0, \dots, N\}$ as Eqs. 1 and 2.

$$\bar{S}_{l,r} = \sum_{i=0}^{\frac{N}{R}-1} s_{i,r} \omega_{\frac{2N}{R}}^{(2l+1)i} \bmod q, \quad (1)$$

$$s_{i,r} = a_{Ri+r} = a_n. \quad (2)$$

- 2) Derive $\{\hat{s}_{l,r}\}$ by multiplying $\{\bar{S}_{l,r}\}$ from Step 1 by $\omega_{2N}^{r(2l+1)}$ as Eq. 3:

$$\hat{s}_{l,r} = \bar{S}_{l,r} \omega_{2N}^{r(2l+1)} \bmod q, \quad (3)$$

- 3) Compute $\frac{N}{R}$ R -point NTT $\{\hat{S}_{l,j} | j = 0, \dots, R-1\}$ ($l = 0, \dots, \frac{N}{R}-1$) of $\{\hat{s}_{l,r} | r = 0, \dots, R-1\}$ ($l = 0, \dots, \frac{N}{R}-1$) from Step 2 as Eq. 4.

$$\hat{S}_{l,j} = \sum_{r=0}^{R-1} \hat{s}_{l,r} \omega_R^{rj} \bmod q, \quad (4)$$

where $\omega_R = \omega_{\frac{2N}{R}} \bmod q$. The final result can be obtained by Eq. 5:

$$\bar{A}_k = \bar{A}_{1+\frac{N}{R}j} = \hat{S}_{l,j}. \quad (5)$$

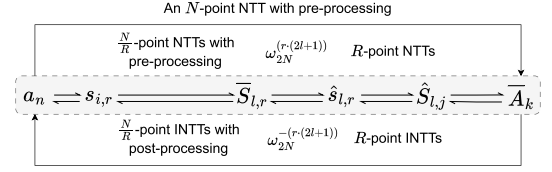
Fig. 2(a) demonstrates the symbols' relationship. Step 3 can be performed by unroll R2NTT as Algorithm 1. For $q = F_n$ and $R \leq 2^{n+1}$, Step 3 requires no multiplications, only shifts, since ω_R is a power of 2, as discussed in Section II-B. By recursively decomposing Step 1 with radix R , the computation complexity can be reduced from $\mathcal{O}(N^2)$ to $\mathcal{O}(N \log_R N)$.

Additionally, one key configuration is to maintain ω_R as the same $\omega_{\frac{2N}{R}}$ from the original size N and ω_{2N} throughout the decomposition. This ensures that twiddle factors remain unchanged across different R -point NTTs, allowing for the design of one fixed R -point NTT module. The upper part of Fig. 2(b) shows the flow of a high-radix NTT with pre-processing, which alternates between multiplications and NTTs. The left part of Fig. 3(a) show an R16 NTT butterfly. In this way, for a Fermat modulus, a NTT with merged pre-processing can leverage power-of-2 constant multiplications without the constraint $N < 2^{n+1}$, only requiring $R \leq 2^{n+1}$.

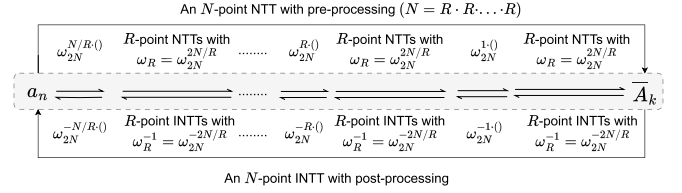
B. High-Radix DIF-INTT With Merged Post-Processing

The radix- R length- N DIF-INTT with merged post-processing can be obtained by

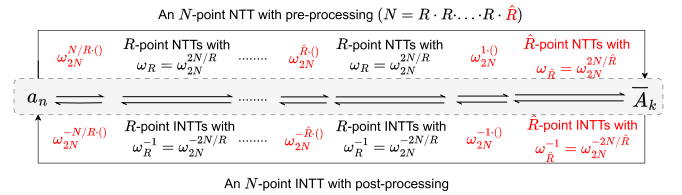
$$\begin{aligned} a_n &= N^{-1} \omega_{2N}^{-n} \sum_{k=0}^{N-1} \bar{A}_k \omega_N^{-kn} \bmod q \\ &= N^{-1} \omega_{2N}^{-n} \left(\sum_{l=0}^{\frac{N}{R}-1} \bar{A}_l \omega_N^{-ln} + \sum_{l=0}^{\frac{N}{R}-1} \bar{A}_{l+\frac{N}{R}} \omega_N^{-(l+\frac{N}{R})n} \right. \\ &\quad + \sum_{l=0}^{\frac{N}{R}-1} \bar{A}_{l+\frac{2N}{R}} \omega_N^{-(l+\frac{2N}{R})n} + \dots \\ &\quad \left. + \sum_{l=0}^{\frac{N}{R}-1} \bar{A}_{l+\frac{(R-1)N}{R}} \omega_N^{-(l+\frac{(R-1)N}{R})n} \right) \bmod q \end{aligned}$$



(a) Symbols relationship in the formulations



(b) High-radix NTT/INTT with pre/post-processing



(c) Mixed-radix NTT/INTT with pre/post-processing

Fig. 2. The flow of high-radix/mixed-radix NTT/INTT with pre/post-processing.

$$= N^{-1} \omega_{2N}^{-n} \sum_{l=0}^{\frac{N}{R}-1} \sum_{j=0}^{R-1} (\bar{A}_{l+\frac{jN}{R}} \omega_R^{-jn}) \omega_N^{-ln} \bmod q.$$

By denoting $\{\bar{A}_{1+\frac{jN}{R}}\}$ as $\{\hat{S}_{l,j}\}$, it can be written as

$$a_n = \left(\frac{N}{R}\right)^{-1} \omega_{2N}^{-n} \sum_{l=0}^{\frac{N}{R}-1} \left(R^{-1} \sum_{j=0}^{R-1} \hat{S}_{l,j} \omega_R^{-jn} \right) \omega_N^{-ln} \bmod q,$$

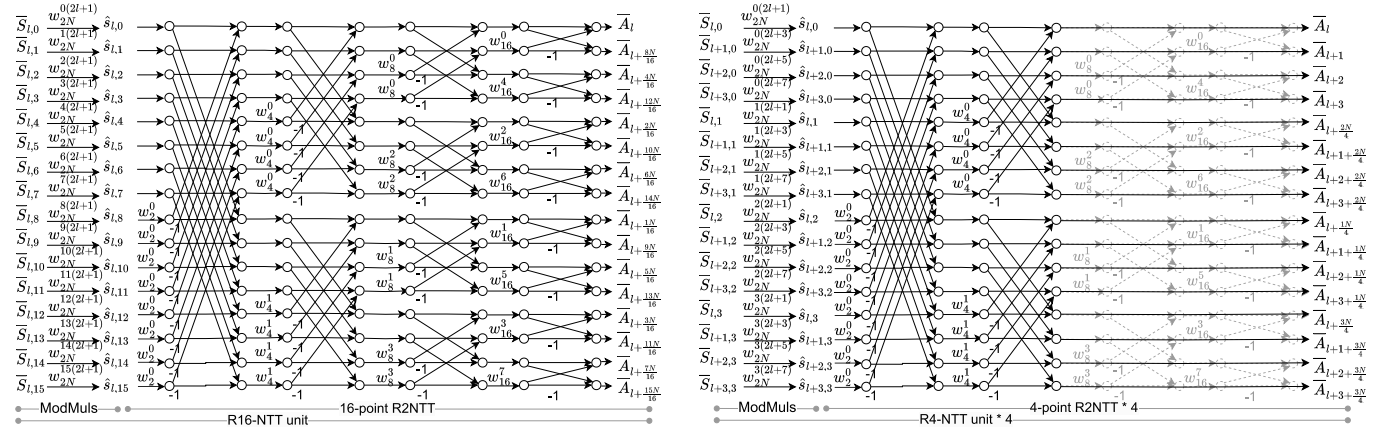
where $k = 0, \dots, N-1$. Due to the periodicity, it can be further expressed as

$$\begin{aligned} a_{Ri+r} &= \left(\frac{N}{R}\right)^{-1} \omega_{2N}^{-(Ri+r)} \sum_{l=0}^{\frac{N}{R}-1} \left(R^{-1} \sum_{j=0}^{R-1} (\hat{S}_{l,j} \omega_R^{-j(Ri+r)}) \right. \\ &\quad \left. \cdot \omega_N^{-l(Ri+r)} \right) \bmod q \\ &= \left(\frac{N}{R}\right)^{-1} \omega_{2N}^{-i} \sum_{l=0}^{\frac{N}{R}-1} \left(\left(R^{-1} \sum_{j=0}^{R-1} \hat{S}_{l,j} \omega_R^{-jr} \right) \right. \\ &\quad \left. \cdot \omega_{2N}^{-(2l+1)r} \right) \cdot \omega_{\frac{N}{R}}^{-li} \bmod q, \end{aligned}$$

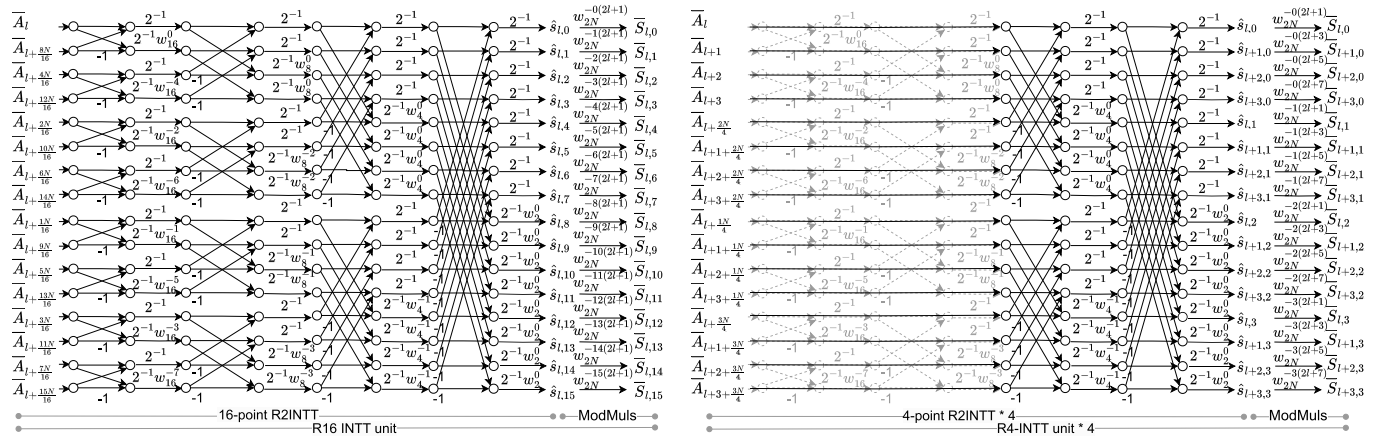
where $i = 0, \dots, \frac{N}{R}-1, r = 0, \dots, R-1$. The steps of INTT with merged post-processing can be summarized as:

- 1) Compute $\frac{N}{R}$ R -point INTT $\{\hat{s}_{l,r} | r = 0, \dots, R-1\}$ ($l = 0, \dots, \frac{N}{R}-1$) of $\{\hat{S}_{l,j} | j = 0, \dots, R-1\}$ ($l = 0, \dots, \frac{N}{R}-1$) from $\{\bar{A}_k | k = 0, \dots, N\}$ as Eqs. 6 and 7 as

$$\hat{s}_{l,r} = R^{-1} \sum_{j=0}^{R-1} \hat{S}_{l,j} \omega_R^{-jr} \bmod q, \quad (6)$$



(a) An R16 DIT-NTT butterfly unit applied to an R16 computation (left) and 4 parallel R4 computations (right).



(b) An R16 DIF-INTT butterfly unit applied to an R16 computation (left) and 4 parallel R4 computations (right).

Fig. 3. R16 butterfly units are applied to R16 cases and R4 cases.

where $\omega_R = \omega_{\frac{2N}{R}} \bmod q$.

$$\hat{S}_{l,j} = \bar{A}_{l+\frac{N}{R}j} = \bar{A}_k, \quad (7)$$

- 2) Derive $\{\bar{S}_{l,r}\}$ by multiplying $\{\hat{S}_{l,r}\}$ from Step 1 by $\omega_{2N}^{-(2l+1)r}$ as Eq. 8:

$$\bar{S}_{l,r} = \hat{S}_{l,r} \omega_{2N}^{-(2l+1)r} \bmod q. \quad (8)$$

- 3) Derive $R \cdot \frac{N}{R}$ -point INTT with merged post-processing $\{s_{i,r} | i = 0, \dots, \frac{N}{R} - 1\}$ ($r = 0, \dots, R - 1$) of $\{\bar{S}_{l,r} | l = 0, \dots, \frac{N}{R} - 1\}$ ($r = 0, \dots, R - 1$) from Step 2 as Eqs. 9 and 10.

$$s_{i,r} = \left(\frac{N}{R}\right)^{-1} \omega_{\frac{2N}{R}}^{-i} \sum_{l=0}^{\frac{N}{R}-1} \bar{S}_{l,r} \omega_{\frac{N}{R}}^{-li} \bmod q, \quad (9)$$

$$a_n = a_{Ri+r} = s_{i,r}, \quad (10)$$

Fig. 2(a) demonstrates the symbols relationship. Step 1 can be performed by an unroll R2INTT as Algorithm 2. For $q = F_n$ and $R \leq 2^{n+1}$, Step 1 consumes no multiplications, only shifts, since ω_R^{-1} is a power of 2, as discussed in Section II-B. By recursively decomposing Step 3 with radix R , the computation complexity can be reduced from $\mathcal{O}(N^2)$ to $\mathcal{O}(N \log_R N)$.

Additionally, one key configuration is to maintain ω_R^{-1} as the same $\omega_{\frac{2N}{R}}$ from the original size N and ω_{2N} throughout the decomposition. This ensures that twiddle factors remain unchanged across different R -point INTTs, allowing for the design of one fixed R -point INTT module. The lower part of Fig. 2(b) shows the flow of a high-radix INTT with post-processing, which alternates between INTTs and modular multiplications. The left part of Fig. 3(b) show an R16 INTT butterfly.

In this way, for a Fermat modulus, an INTT with merged post-processing can leverage power-of-2 constant multiplications without the constraint $N < 2^{n+1}$, only requiring $R \leq 2^{n+1}$.

C. Proposed High-Radix/Mixed-Radix Algorithms for DIT-NTT and DIF-INTT

In this subsection, we propose an improved iterative algorithms for high-radix and mixed-radix NTT and INTT. The iterative algorithms for high-radix/mixed-radix NTT with merged pre-processing and INTT with merged post-processing are detailed in Algorithms 4 and 5, respectively. To clarify, in this paper, one complete process of ModMuls and an R2NTT/R2INTT

Algorithm 4: High-radix/Mixed-radix NTT_{NR} with merged pre-processing**Input :** A vector a of length N , modulus q , $2N$ -th primitive root of unity ω_{2N} in $\mathbb{Z}_q$, radix R .**Output:** Output vector $\bar{A}$.

```

1   $\omega_R \leftarrow \omega_{2N}^{2N/R}$ ;
2  for  $s \leftarrow 0$  to  $\lfloor \log(N)/\log(R) \rfloor - 1$  do
3       $M \leftarrow R^{s+1}$ ;
4       $\omega_{2M} \leftarrow \omega_{2N}^{N/M} \bmod q$ ;
5       $\Delta_{\text{idx}} \leftarrow N/R^{s+1}$ ;
6      for  $b \leftarrow 0$  to  $R^s - 1$  do
7          for  $g \leftarrow 0$  to  $N/R^{s+1} - 1$  do
8               $\text{idx} \leftarrow b \cdot N/R^s + g$ ;
9              // unroll from line 9 to 14
10             for  $r \leftarrow 0$  to  $R - 1$  do
11                  $T_r \leftarrow a_{\text{idx} + \Delta_{\text{idx}} \cdot r} \cdot \omega_{2M}^{r \cdot (2 \cdot \text{bit-rev}(b) + 1)} \bmod q$ 
12                 // ModMul
13                  $T_r \leftarrow \text{Norm\_to\_D1}(T_r)$ ;
14              $T \leftarrow \text{R2NTT}(T, R, q, \omega_R)$ 
15             for  $r \leftarrow 0$  to  $R - 1$  do
16                  $a_{\text{idx} + \Delta_{\text{idx}} \cdot r} \leftarrow \text{D1\_to\_Norm}(T_r)$ ;
17
18  $s \leftarrow s + 1$ ;
19
20 if  $s \neq \log(N)/\log(R)$  then
21      $\hat{R} \leftarrow N/R^s$ ;
22      $\omega_{2M} \leftarrow \omega_{2N}$ ;
23     for  $b \leftarrow 0$  to  $R^s - 1$  by  $R/\hat{R}$  do
24          $\text{idx} \leftarrow b \cdot \hat{R}$ ;
25         // unroll from line 20 to 27
26         for  $r1 \leftarrow 0$  to  $\hat{R} - 1$  do
27             for  $r2 \leftarrow 0$  to  $\frac{R}{\hat{R}} - 1$  do
28                  $T_{r1 \cdot \frac{R}{\hat{R}} + r2} \leftarrow a_{\text{idx} + r1 + r2 \cdot \hat{R}} \cdot \omega_{2M}^{r1 \cdot (2 \cdot \text{bit-rev}(b + r2) + 1)}$ 
29                  $\bmod q$  // ModMul
30                  $T_{r1 \cdot \frac{R}{\hat{R}} + r2} \leftarrow \text{Norm\_to\_D1}(T_{r1 \cdot \frac{R}{\hat{R}} + r2})$ ;
31
32  $T \leftarrow$  The first  $\log \hat{R}$  substages of R2NTT( $T, R, q, \omega_R$ );
33 // Algorithm 1 with line 1 having  $s$  from
34 0 to  $\log \hat{R} - 1$ 
35 for  $r1 \leftarrow 0$  to  $\hat{R} - 1$  do
36     for  $r2 \leftarrow 0$  to  $\frac{R}{\hat{R}} - 1$  do
37          $a_{\text{idx} + r1 + r2 \cdot \hat{R}} \leftarrow \text{D1\_to\_Norm}(T_{r1 \cdot \frac{R}{\hat{R}} + r2})$ ;
38
39  $\bar{A} \leftarrow a$ ;

```

is called a ‘stage’, while a single layer of R2 butterfly operations within the R2NTT is termed a ‘substage’. Our optimization can be captured in three main points.

- **First, we maximize the concentration of power-of-2 constant multiplications into the fixed R2NTT and R2INTT using high-radix methods.**

By setting $q = F_n$ and $R \leq 2^{n+1}$, all the twiddle factors in R2NTT/R2INTT are powers of 2 because ω_R is a power of 2 as discussed in Section II-B. The R2NTT/R2INTT is static in different stages due to a fixed ω_R . It is important to note that there can be multiple R -th roots of unity or N -th roots of unity in $\mathbb{Z}_q$. However, ω_R must be $\omega_{2N}^{2N/R}$ for the selected ω_{2N} to ensure correctness.

In other designs, although many twiddle factors are powers of 2 at various stages (especially $\omega^0 = 1$), these twiddle factors are not fixed in the butterfly module, requiring flexible multipliers to handle different twiddle factors.

In contrast, our algorithm separates static power-of-2 multiplications in R2NTT/R2INTT and dynamic non-power-of-2 multiplications in ModMuls. This allows specific constant processing to be applied to R2NTT/R2INTT, and the ModMuls hardware can be reused for PWM between NTT and INTT.

- **Second, we reuse the R -point R2NTT and R2INTT for the special radix- $\hat{R}$ stage in mixed-radix cases.**

To address cases where R is not a root of N , we propose a mixed-radix approach by applying a radix- $\hat{R}$ stage at the end of NTT and the beginning of INTT, as illustrated in Fig. 2(c). Fig. 3(a) and 3(b) demonstrate how to extract four R4 butterfly unit from an R16 unit in NTT and INTT,

respectively. The ModMuls is fully reused, while the first $\log \hat{R}$ substages of radix- R R2NTT and the last $\log \hat{R}$ substages of radix- R R2INTT are reused to construct radix- $\hat{R}$ R2NTT and R2INTT respectively.

- **Third, we strategically apply D1 representation in R2NTT and R2INTT and normal representation in ModMuls.**

The D1 representation exhibits varying levels of efficiency for different arithmetic operations in $\mathbb{Z}_{F_n}$, as illustrated in Algorithm 3. Modular multiplications by a power of 2 benefit the most, requiring only circular shifting. Modular addition offers neither significant benefits nor disadvantages, while modular subtraction is similar to addition since it can be treated as a negation followed by a modular addition.

However, modular multiplication by non-power-of-2 numbers in D1 is more complex. The direct computation is illustrated as $D1(x \cdot \omega) \bmod q = x \cdot \omega - 1 \bmod q = (x - 1) \cdot (\omega - 1) + x + \omega - 2 \bmod q = D1(x) \cdot D1(\omega) + D1(x) + D1(\omega) \bmod q$. Instead, [5] computes normal multiplication by computing the partial products $2^i x[i] \cdot \omega$ with D1, followed by accumulation in D1. While this method leverages the benefits of D1, it sacrifices DSP utilization in FPGA. Applying this method in high-radix cases would lead to significant consumption of LUTs and FFs, along with potential frequency degradation due to complex routing.

Therefore, we choose to apply D1 representation in the R2NTT/R2INTT, which involves modular multiplications by power-of-2 constants, modular additions and modular

Algorithm 5: High-radix/Mixed-radix INTT_{RN} with merged post-processing

Input : A vector $\bar{A}$ of length N , modulus q , the inversion of a $2N$ -th primitive root of unity ω_{2N}^{-1} in $\mathbb{Z}_q$, radix R .
Output : Output vector a .

```

1  $\omega_R^{-1} \leftarrow (\omega_{2N}^{-1})^{2N/R}$ ;
2  $s \leftarrow \lfloor \log(N)/\log(R) \rfloor$ ;
3 if  $s \neq \log(N)/\log(R)$  then
4    $\hat{R} \leftarrow N/(R^s)$ ;
5    $\omega_{2M}^{-1} \leftarrow \omega_{2N}^{-1}$ ;
6   for  $b \leftarrow 0$  to  $R^s - 1$  by  $R/\hat{R}$  do
7      $\text{idx} \leftarrow b \cdot \hat{R}$ ;
8     // unroll from line 8 to 16
9     for  $r1 \leftarrow 0$  to  $\hat{R} - 1$  do
10      for  $r2 \leftarrow 0$  to  $\frac{R}{\hat{R}} - 1$  do
11        $T_{r1 \cdot \frac{R}{\hat{R}} + r2} \leftarrow \text{Norm\_to\_D1}(\bar{A}_{\text{idx} + r1 + r2 \cdot \hat{R}})$ ;
12        $T \leftarrow$  The last  $\log \hat{R}$  substages of  $\text{R2INTT}(T, R, q, \omega_R^{-1})$ ;
13       // Algorithm 2 with line 1 having  $s$  from
14       //  $\log \hat{R} - 1$  to 0
15       for  $r1 \leftarrow 0$  to  $\hat{R} - 1$  do
16       for  $r2 \leftarrow 0$  to  $\frac{R}{\hat{R}} - 1$  do
17         $T_{r1 \cdot \frac{R}{\hat{R}} + r2} \leftarrow \text{D1\_to\_Norm}(T_{r1 \cdot \frac{R}{\hat{R}} + r2})$ ;
18         $T_{r1 \cdot \frac{R}{\hat{R}} + r2} \leftarrow \bar{A}_{\text{idx} + r1 + r2 \cdot \hat{R}} \cdot \omega_{2M}^{-r1 \cdot (\frac{R}{\hat{R}} - \text{bit-rev}(b+r2) + 1)}$ 
19         $\bar{A}_{\text{idx} + r1 + r2 \cdot \hat{R}} \leftarrow T_{r1 \cdot \frac{R}{\hat{R}} + r2} \bmod q$ 
20        // ModMul
21
22 for  $s \leftarrow \lfloor \log(N)/\log(R) \rfloor - 1$  to 0 do
23    $M \leftarrow R^{s+1}$ ;
24    $\omega_{2M}^{-1} \leftarrow (\omega_{2N}^{-1})^{N/M} \bmod q$ ;
25    $\Delta \text{idx} \leftarrow N/R^{s+1}$ ;
26   for  $b \leftarrow 0$  to  $R^s - 1$  do
27     for  $g \leftarrow 0$  to  $N/R^{s+1} - 1$  do
28        $\text{idx} \leftarrow b \cdot N/R^s + g$ ;
29       // unroll from line 24 to 29
30       for  $r \leftarrow 0$  to  $R - 1$  do
31         $T_r \leftarrow \text{Norm\_to\_D1}(\bar{A}_{\text{idx} + \Delta \text{idx} \cdot r})$ ;
32         $T \leftarrow \text{R2INTT}(T, R, q, \omega_R^{-1})$  for  $r \leftarrow 0$  to  $R - 1$  do
33         $T_r \leftarrow \text{D1\_to\_Norm}(T_r)$ ;
34         $\bar{A}_{\text{idx} + \Delta \text{idx} \cdot r} \leftarrow T_r \cdot \omega_{2M}^{-r \cdot (2 \cdot \text{bit-rev}(b) + 1)}$  mod  $q$ 
35        // ModMul
36
37  $a \leftarrow \bar{A}$ ;

```

subtractions. For the ModMuls, which only involves modular multiplications by normal numbers, we use the normal representation. This approach optimizes DSP resource usage in FPGAs while maintaining the advantages of D1 for modular multiplications by powers of 2.

IV. REDUCED-COMPLEXITY CONFLICT-FREE MEMORY MAPPING

In an in-place design, R parallel data are read from R banks, processed, and then written back to the same addresses. Existing designs require R parallel address generators to compute the bank indexes, which control the interconnection of the bank addresses and the data read/write. By using the method discussed in Section II-C, the existing mapping algorithm can be summarized in Algorithm 6. In previous works, R is small so it is not a heavy task to compute R bank indexes in parallel. However, for larger R (e.g. $R = 16$), calculating and keeping R parallel bank indexes to select signals across the whole architecture will degrade system efficiency in terms of resources and frequency. Our mapping scheme addresses this issue by using only one address generator as well as one select signal.

The proposed mapping scheme is come up from an observation. As shown in Fig. 1(a), data with original addresses are partitioned into four banks. The bank indexes are indicated by different colors. By a careful check on Fig. 1(a) and 1(d), one could observe that the bank indexes in one particular cycle are

Algorithm 6: Existing memory mapping scheme

AddressGeneration

```

for  $j \leftarrow 0, \dots, R - 1$  do
   $\text{BankIndex}_j \leftarrow \sum_{i=0}^{\lceil \frac{\log N}{\log R} \rceil - 1} \text{OrigAddr}_j[(\log R) \cdot (i + 1) : (\log R)i] \bmod R$ ;

```

InterconnectBankOut

```

 $\text{Op}_{0,1,\dots,R-1} \leftarrow$  data from  $\text{BankIndex}_{0,R-1}$ ;

```

InterconnectBankAddr

```

 $\text{Addr with BankIndex}_{0,R-1} \leftarrow \text{OrigAddr}_{0,R-1}[\log N - 1 : \log R]$ ;

```

InterconnectBankIn

```

Data with  $\text{BankIndex}_{0,R-1} \leftarrow$  Op with index 0, 1, ...,  $R - 1$ ;

```

correlated. That is, the bank indexes of operands 1, 2 and 3 can be inferred when the bank index of operand 0 is known. In single-radix cases, the data out of the banks can be arranged by one-bit circular shifting, resulting in only R possible cases of arranging data from banks.

Fig. 4 depicts the proposed data mapping in a 64-point R4 design. Data from the four banks are reordered to four circular shifting groups, which are from {Bank0, Bank1, Bank2, Bank3}, {Bank1, Bank2, Bank3, Bank0}, {Bank2, Bank3, Bank0, Bank1} and {Bank3, Bank0, Bank1, Bank2}. The output of the multiplexer is 4 parallel data from one group of these choices, selected by the calculated bank index of the first operand. After performing the main calculation, the

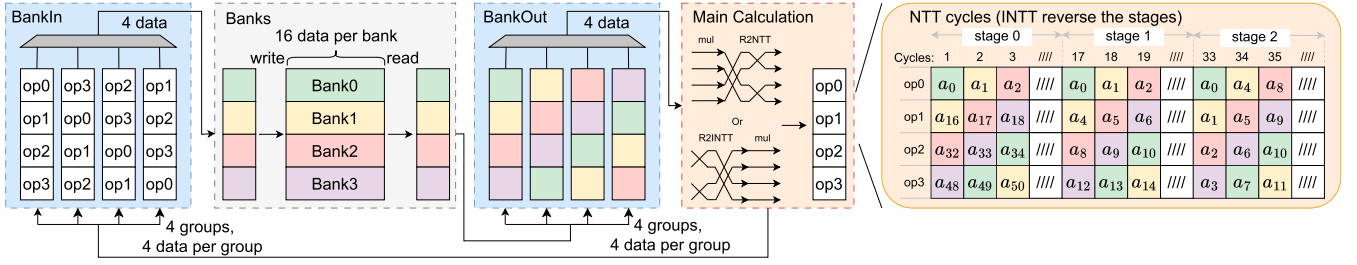


Fig. 4. Conflict-free memory mapping for 64-point high-radix R4 NTT/INTT.

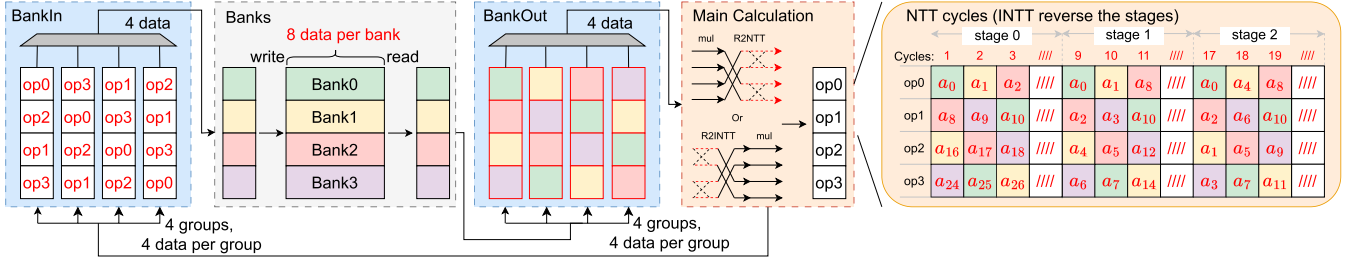


Fig. 5. Conflict-free memory mapping for 32-point mixed-radix (R4&R2) NTT/INTT. The modified part compared to Fig. 4 is highlighted in red color. The butterflies are either fully applied for R4 stages or partially applied for the R2 stage.

four operands are reordered to the four corresponding reverse groups, which are {Operand0, Operand1, Operand2, Operand3}, {Operand3, Operand0, Operand1, Operand2}, {Operand2, Operand3, Operand0, Operand1} and {Operand1, Operand2, Operand3, Operand0}. Selected by the same bank index, the output of the multiplexer is then fed into the banks.

In mixed-radix cases, the data arrangement is a little bit different. Fig. 5 shows our data mapping in a 32-point R4&R2 NTT or INTT. We apply a full R4 butterfly for the first two stages and the first substage of the butterfly for the last stage of NTT. For INTT, the stages are in reverse. It is worth noting that in the 2*R2 design, like [24], the two R2 butterflies are parallel and separate so that their {Operand0, Operand1} and {Operand2, Operand3} are fed into two R2 butterflies respectively. However, the two R2 in our R4 butterfly here for the last stage is across to each other, so that the {Operand0, Operand2} and {Operand1, Operand3} are fed into two R2 butterflies respectively. In conclusion, the required order for the bank indexes of four operands has 4 possible groups again, which is {Bank0, Bank2, Bank1, Bank3}, {Bank1, Bank3, Bank2, Bank0}, {Bank2, Bank0, Bank3, Bank1} and {Bank3, Bank1, Bank0, Bank2}. The corresponding writing orders for each one are {Operand0, Operand2, Operand1, Operand3}, {Operand3, Operand0, Operand2, Operand1}, {Operand1, Operand3, Operand0, Operand2} and {Operand2, Operand1, Operand3, Operand0}.

To generalize, a new mapping scheme for single-radix and mixed-radix is summarized in Algorithm 7. The bank indexes and operand indexes are precomputed. The data, addresses and operands are in R possible orders respectively, waiting for the

Algorithm 7: Proposed memory mapping scheme

```

Precomputed
//  $\hat{R}=1$  when it is single radix.
for  $i \leftarrow 0, \dots, \frac{R}{\hat{R}} - 1$  do
    for  $j \leftarrow 0, \dots, \hat{R} - 1$  do
        for  $k \leftarrow 0, \dots, R - 1$  do
            BankIndexes $_{i+j \cdot \frac{R}{\hat{R}}, k} \leftarrow i \cdot \hat{R} + j + k \bmod R$ ;
for  $i \leftarrow 0, \dots, \frac{R}{\hat{R}} - 1$  do
    for  $j \leftarrow 0, \dots, \hat{R} - 1$  do
        for  $k \leftarrow 0, \dots, R - 1$  do
            OpIndexes $_{(i \cdot \hat{R} + j + k \bmod R), k} \leftarrow i + j \cdot \frac{R}{\hat{R}}$ ;
AddressGeneration
[ $\frac{\log N}{\log R}$ ]-1
iSelect  $\leftarrow \sum_{i=0}^{[\frac{\log N}{\log R}]-1} \text{OrigAddr}_0[(\log R) \cdot (i+1) : (\log R)i]$ 
mod  $R$ ;
InterconnectBankOut
Op $_{0:R-1} \leftarrow \text{data from BankIndexes}_{0:R-1, iSelect}$ ;
InterconnectBankAddr
Addr $_{0:R-1} \leftarrow \text{OrigAddr}[\log N - 1 : \log R]$ 
with OpIndexes $_{0:R-1, iSelect}$ 
InterconnectBankIn
Data $_{0:R-1} \leftarrow \text{Op with OpIndexes}_{0:R-1, iSelect}$ 

```

selection by the **iselect** signal which is the bank index of the first operand calculated by one address generator. Only one required group for each interconnection part is selected in each

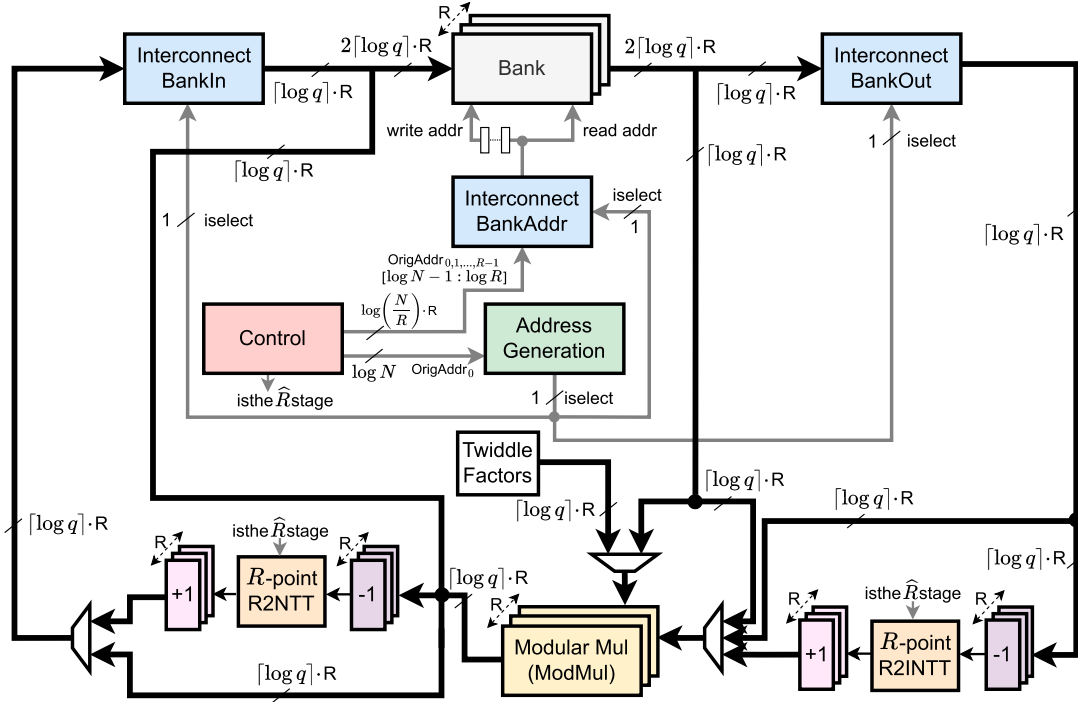


Fig. 6. An overall polynomial multiplication architecture based on high-radix/mixed-radix in-place NTT over fermat modulus. “−1” denotes the conversion from normal representation to D1 representation. “+1” denotes the conversion from D1 representation to normal representation.

selection. This scheme has lower complexity by minimizing address generators and select signals for interconnection modules to one, enhancing resource efficiency and frequency in high-radix/mixed-radix design.

V. HIGH-RADIX/MIXED-RADIX NTT MULTIPLICATION ARCHITECTURE

A. The Overall Architecture

Fig. 6 depicts the overall architecture of the proposed design, which is fit for the proposed algorithms. This architecture supports complete polynomial multiplication, where the process consists of the first NTT, the second NTT, one PWM, and one INTT, executed sequentially.

In each cycle, the control module provides the original addresses, denoted as **OrigAddr**. Specifically, the original addresses are R indexes of parallel processing data of the sequence in Algorithm 4 for NTT and Algorithm 5 for INTT. For the PWM procedure between two NTTs and one INTT, the original addresses are provided by the first stage of INTT. The **OrigAddr**₀ is fed into address generation to compute the signal **islect**, which is the key of our reduced-complexity memory mapping scheme. The signal **islect** is fed into three interconnection modules to determine the data selection and address selection.

According to the indexes and addresses, R parallel data are derived from R banks. In this design, the coefficients of two polynomials are aligned one by one and stored in the lower halves and upper halves of the same addresses respectively, thus the bit-width of each data is twice the size of modulus.

TABLE II
DETAILED STATUS FOR WRITING TO/READING FROM BANK

State	Signal	To/From Bank	Supported Procedure
Write	11	All	Input, PWM
	01	Lower Half	NTT1, PWM, INTT
	10	Upper Half	NTT2
Read	11	All	PWM
	01	Lower Half	NTT1, INTT, Output
	10	Upper Half	NTT2

TABLE III
COMPARISON OF SUPPORTED PARAMETERS AND OPERATIONS IN STATE-OF-THE-ART DESIGNS AND OUR PROPOSED DESIGN

Work	q	R	N	Supported Op.
[5]	Fermat	2	256	(I)NTT/PWM
[21]	tunable	2, 4^M	scalable	(I)NTT/PWM
[25]	tunable	2	scalable	(I)NTT/PWM
[17]	tunable	2, 4	scalable (R2) limited (R4)	(I)NTT/PWM
[24]	fixed	2	512, 1024	(I)NTT
[16]	tunable	2, 4	1024	(I)NTT
Ours	Fermat	2, 4, 8, 16^M	scalable	(I)NTT/PWM

^M The design supports the mixed-radix setting.

When performing PWM, the lower halves and upper halves of data would be fed into the ModMul modules. The product of them would be written as the lower halves of data into the banks. When computing NTT or INTT, the lower halves (for

TABLE IV
IMPLEMENTATION RESULTS OF POLYNOMIAL MULTIPLICATION (2 NTTs, 1 PWM AND 1 INTT) ON FPGA AND COMPARISONS

Work	Device	modulus q	BFU ^a	Radix ^b	LUT /ATP	FF /ATP	DSP /ATP	BRAM /ATP	Freq. (MHz)	Cycles	Time (μ s)
$N = 256$											
[5] ^c	Spartan-6	65537 (17-bit)	1*R2	2	404/5.4	599/8.0	0/0.0	1/13.3	287	3824	13.3
[21]	Zynq-7	3329 (12-bit)	28*R2,1*R4	4, 2	9211/8.6	9810/9.1	60/55.7	1/0.9	265	246	0.9
[25] ^d	Kintex-US+	12289 (14-bit)	16*R2	2	24186/41.2	14756/25.1	48/81.8	24/40.9	200	341	1.7
[17]	Virtex-7	13-bit	1*R2	2	449/5.3	271/3.2	3/35.2	3/35.2	286	3354	11.7
			1*R4	4	1288/4.1	888/2.8	12/37.9	4.5/14.2	278	879	3.2
			8*R2	2	6245/10.8	1864/3.2	24/41.4	12/20.7	256	442	1.7
Ours	Virtex-7	65537 (17-bit)	1*R4	4	1690/4.9	1289/3.7	4/11.6	0/0.0	301	876	2.9
			1*R8	8, 4	3596/4.6	2888/3.7	8/10.3	0/0.0	301	389	1.3
			1*R16	16	8078/5.8	6184/4.4	16/11.5	0/0.0	274	197	0.7
$N = 512$											
[21]	Zynq-7	12289 (14-bit)	32*R2,1*R4	4, 2	19103/33.2	13677/23.8	68/118.3	1/1.7	261	454	1.7
[24] ^e	Zynq-7	12289 (14-bit)	2*R2	2	741/12.1	330/5.4	2/32.7	5/81.8	245	4010	16.4
[17]	Virtex-7	14-bit	1*R2	2	489/13.1	245/6.6	3/80.4	3/80.4	278	7450	26.8
			16*R2	2	28616/91.1	4211/13.4	48.0/152.7	24/76.4	154	490	3.2
Ours	Virtex-7	65537 (17-bit)	1*R4	4, 2	1955/13.6	1286/8.9	4/27.8	2/13.9	301	2092	6.9
			1*R8	8	3559/8.2	2737/6.3	8/18.5	3.5/8.1	301	697	2.3
			1*R16	16, 2	8904/13.1	6321/9.3	16/23.5	0/0.0	274	402	1.5
$N = 1024$											
[21]	Zynq-7	12289 (14-bit)	36*R2,1*R4	4, 2	23261/76.6	14901/49.1	76/250.2	1/3.3	257	846	3.3
[24] ^e	Zynq-7	12289 (14-bit)	2*R2	2	847/27.6	375/12.2	2/65.3	6/195.8	244	7964	32.6
[16] ^e	Virtex-7	14-bit	1*R4	4	1196/18.5	969/15.0	12/186.0	3/46.5	270	4186	15.5
			2*R4	4	2953/25.2	1875/16.0	27/230.9	5.5/47.0	250	2138	8.6
			1*R2	2	475/27.2	307/17.6	3/171.7	1.5/85.9	278	15915	57.2
			8*R2	2	6300/56.3	2124/19.0	27/241.1	10/89.3	227	2027	8.9
[25] ^d	Kintex-US+	12289 (14-bit)	16*R2	2	22648/132.3	15030/87.8	48/280.5	24/140.2	200	1169	5.8
[17]	Virtex-7	14-bit	1*R4	4	1467 / 32.2	930 / 20.4	13 / 285.0	4.5 / 98.6	189	4143	21.9
			4*R4	4	8515 / 53.0	3618 / 22.5	49 / 305.1	12 / 74.7	172	1071	6.2
Ours	Virtex-7	65537 (17-bit)	1*R4	4	1395/19.2	1222/16.8	4/55.0	4/55.0	301	4140	13.7
			1*R8	8, 2	3463/19.7	2665/15.1	8/45.5	7.5/42.6	301	1712	5.7
			1*R16	16, 4	9783/25.6	6537/17.1	16/41.8	0/0.0	274	716	2.6

^a BFU: The style of butterfly unit for one NTT. For example, 2*R4 indicates two parallel radix-4 units in the NTT design.

^b If two radix values are listed, this indicates mixed-radix. In our work, mixed-radix is computed by designing butterfly unit architecture of higher radix, where the lower radix is computed by using part of the same architecture.

^c The design supports only one NTT, one point-wise multiplication and one INTT. One more NTT is added to the cycles and time, assuming the given cycles and time from the work is $(2 + \frac{1}{\#stages})$ NTT. The area is assumed to remain the same.

^d The work does not list cycles of this configuration (q and N). The cycles and time here is approximated by PWM cycles in a 16*R2 configuration ($q = 8380417$, $N = 256$) times the ratio of NTT cycles between the two configurations.

^e The design supports only one NTT/INTT. The cycles and time here is approximated by that of $(3 + \frac{1}{\#stages})$ NTT, while the area is assumed to remain the same.

the first NTT and the INTT) or the upper halves (for the second NTT) are reordered by the interconnection module. For NTT, the data are routed to the ModMul modules, followed by the R -point R2NTT module for further processing. For INTT, the data are first sent to the R -point R2INTT module, and then to the ModMul modules for computation. The outputs pass through another interconnection module before being written back to the memory banks. The signal **istheRstage** indicates whether the special $\hat{R}$ stage is active, determining if the R2NTT and R2INTT modules should be fully applied or used partially in their specific substages.

Table II summarizes operations and support procedures for different write states and read states.

VI. IMPLEMENTATION RESULTS AND COMPARISONS

The proposed architecture is designed using Verilog HDL and implemented on the 28nm Xilinx Virtex-7 FPGA (xc7vx690tffg1761-3). Vivado 2022.2 is used for synthesis and implementation. Nine different settings are implemented, with $N = 256, 512, 1024$ and $R = 4, 8, 16$. For cases where N is not a power of R , the related mixed-radix design is applied.

Table III provides a general analysis, while Table IV presents the detailed implementation results of our proposed polynomial multiplication design compared to state-of-the-art designs. We assume that a complete polynomial multiplication consists of 2 NTTs, 1 PWM and 1 INTT. However, some of the compared designs, such as [5], [16], [24], do not support a complete polynomial multiplication. For these designs, we assume that the INTT requires the same number of cycles as the NTT, and that the PWM requires the same number of cycles as a single stage of the NTT. We do not consider additional resource consumption or potential frequency degradation, which are very likely to occur due to increased data storage requirements and more complex routing. We believe that this assumption is generous and lenient for designs that do not fully support polynomial multiplication.

Firstly we compare our work with [5], which uses the same Fermat modulus $q = 65537$. Ma et al. [5] propose an R2-NTT polynomial multiplication design with one R2 butterfly for a fixed degree $N = 256$. Our R4/R8/R16 designs consume only 22.9%/10.2%/5.2% cycles of their work, primarily due to the benefits of higher radix. Since Ma et al. [5] employ a modular multiplier designed with shifting and accumulation using D1 representation, their design does not use DSPs but relies on a relatively large number of LUTs and FFs. Our work achieves around 50% better area-time product (ATP) in terms of FFs. Additionally, by employing LUT-based memory instead of BRAM in cases where a large R and a small N lead to very small storage requirements per bank, our R8/R16 designs finally achieve a slightly better ATP in terms of LUTs while eliminating all BRAM usage.

Then we compare our work with [21] which features a mixed-radix design. Duong-Ngoc and Lee [21] implement a fully pipelined design without reordering buffers. Their NTT architecture consists of 4 sequences of R2 butterfly units in the initial stages, followed by a R4 butterfly unit at the end, resulting in a mixed-radix architecture combining R2 and R4 for each degree. For the most similar configurations in $N = 512$, the proposed mixed-radix R4&R2 design uses $2.6\times$ cycles more than [21]. This difference is primarily due to two factors: their design includes two NTTs in parallel, and they use a large quantity of butterfly units to construct a pipeline across stages. Additionally, they use only one BRAM because no temporary storage is required. However, these advantages come at the cost of significantly higher LUT, FF, and DSP usage. In contrast, our work efficiently reuses butterfly units, resulting in 59.0%/62.6%/76.5% less ATP in terms of LUT/FF/DSP. For additional configurations, such as those with a higher radix like R16, our design can achieve faster performance while using fewer resources in every category compared to [21], demonstrating even greater efficiency gain, despite their modulus bit-width being 3–5 bits smaller than ours.

Lastly we compare representative results from [16], [17], [24], [25], which propose architectures with various numbers of R2 or R4 butterfly units. Generally, as observed in [16] and [17], R4 is more efficient than R2 for a single butterfly unit, but efficiency decreases with the number of butterfly units. Although all the selected results in [16], [17], [24], [25] are on modulus

with smaller bit-width, our work generally outperforms them in ATPs of DSP/BRAM while remaining similarly competitive in those of LUT/FF.

Zhang et al. [24] propose a 2*R2 NTT design on a 14-bit modulus 12289. For $N = 512$, our R8 design achieves 43.4%/90.1% less ATP in terms of DSP/BRAM. For $N = 1024$, our R8&R2 design achieves 30.3%/78.2% less ATP in terms of DSP/BRAM. Chen et al. [16] propose NTT design with various numbers of parallel R2 or R4 butterflies on a tunable 14-bit modulus and $N = 1024$. Compared with their best configuration, i.e., 1*R4, our R8&R2 design achieves 75.6%/8.4% less ATP in terms of DSP/BRAM but similar ATP in terms of LUT/FF. Li et al. [25] propose an NTT-based polynomial multiplication design with different numbers of parallel R2 butterflies on tunable modulus and scalable degree. The butterfly unit performs NTT, PWM, and INTT according to the control signal. Their design with the best performance uses 16*R2 units. Due to their large numbers of parallel processing elements, our 1*R16 design achieves more than 80% less ATP across all categories for both $N = 256$ and 1024. Mu et al. [17] propose a scalable NTT-based polynomial multiplication design with different numbers of parallel R2 or R4 butterflies on tunable modulus and scalable degree. As 512 is not a power of 4, Mu et al. [17] do not support R4 in $N = 512$. Comparing with their best results, our work achieves 70%–85% less ATP in terms of DSP and over 90% less ATP in terms of BRAM for $N = 256/512/1024$. The above analysis demonstrates the great potential of the Fermat modulus combined with our proposed improvement methods.

VII. CONCLUSION AND FUTURE WORKS

In summary, a highly efficient polynomial multiplication architecture based on high-radix and mixed-radix NTT over Fermat modulus is proposed. The detailed derivation process for high-radix and mixed-radix NTT with merged pre-processing and INTT with merged post-processing is provided. To facilitate PWM calculations and to leverage the complexity advantages of both normal and D1 representations, the NTT/INTT processes have been modularized into ModMuls and R2NTT/R2INTT modules. To maintain efficiency across a broader range of configurations, R2NTT/R2INTT is reused in the final stage of mixed-radix cases. Additionally, the complexity of the memory mapping scheme is reduced to improve frequency in high-radix cases. In most configurations, our design reduces DSP ATP by approximately 30%–85% and BRAM ATP by 70%–100%, while remaining competitive in LUT and FF ATP. While a few designs may show advantages in specific aspects, our approach consistently demonstrates strong overall efficiency.

Looking ahead, our design can serve as a building block for constructing efficient implementations within existing schemes such as Hawk. The efficiency of FNT-based polynomial multiplication on alternative platforms—such as GPU or ASIC—remains an area for further investigation. Additionally, exploring more FHE or PQC schemes that can support Fermat moduli presents a challenging yet promising research opportunity. Moreover, the design and optimization of polynomial

multiplication over a broader class of moduli—such as Mersenne, pseudo-Mersenne/Fermat and even undiscovered forms with more favorable arithmetic properties—remain open research directions. It is anticipated that the strategic selection and use of such moduli will lead to more elegant and computationally efficient cryptographic constructions.

REFERENCES

- [1] Q. D. Truong, P. Duong-Ngoc, and H. Lee, "Hybrid number theoretic transform architecture for homomorphic encryption," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 33, no. 7, pp. 2039–2043, Jul. 2025.
- [2] Z. Ye et al., "PQNTRU: Acceleration of NTRU-based schemes via customized post-quantum processor," *IEEE Trans. Comput.*, vol. 74, no. 5, pp. 1649–1662, May 2025.
- [3] Z. Ye, R. Song, H. Zhang, D. Chen, R. C. Cheung, and K. Huang, "A highly-efficient lattice-based post-quantum cryptography processor for IoT applications," *IACR Trans. Cryptogr. Hardware Embedded Syst.*, vol. 2024, no. 2, pp. 130–153, 2024.
- [4] A. Kim et al., "Exploring the advantages and challenges of Fermat NTT in FHE acceleration," in *Proc. Annu. Int. Cryptol. Conf.*, Cham, Switzerland: Springer, 2024, pp. 76–106.
- [5] L. Ma, X. Wu, and G. Bai, "A low cost high performance polynomial multiplier design for FPGA implementation," in *Proc. IEEE 3rd Int. Conf. Electron. Technol. (ICET)*, Piscataway, NJ, USA: IEEE Press, 2020, pp. 83–86.
- [6] A. Abdulrahman, V. Hwang, M. J. Kannwischer, and A. Sprenkels, "Faster kyber and dilithium on the cortex-m4," in *Proc. Int. Conf. Appl. Cryptogr. Netw. Secur.*, Cham, Switzerland: Springer, 2022, pp. 853–871.
- [7] L. Leibowitz, "A simplified binary arithmetic for the Fermat number transform," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 24, no. 5, pp. 356–359, Oct. 1976.
- [8] L. Lucas, E. W. Postlethwaite, and W. van Woerden, "HAWK: Module LIP makes lattice signatures fast, compact and simple," in *Proc. 28th Int. Conf. Theory Appl. Cryptol. Inf. Secur. Adv. Cryptol. (ASIACRYPT)*, Taipei, Taiwan, vol. 13794, Cham, Switzerland: Springer Nature, 2023, p. 65.
- [9] Y. Xing et al., "Low-complexity chromatic dispersion compensation using high-radix Fermat number transform," *J. Lightw. Technol.*, vol. 42, no. 15, pp. 5190–5203, 2024.
- [10] Z. Liu, S. Chen, J. Chen, Y. Li, and M. Tang, "Low complexity equalization in coherent optical communication based on Fermat number transform," *Opt. Lett.*, vol. 49, no. 14, pp. 3910–3913, 2024.
- [11] S. Chen et al., "Fermat number transform based chromatic dispersion compensation and adaptive equalization algorithm," 2024, *arXiv:2405.04253*.
- [12] A. Daher, E. H. Baghious, N. El Khouja, E. Radoi, and G. Burel, "Fast algorithm for optimal design of Fermat number transform based block digital filters," *Digit. Signal Process.*, vol. 113, Jun. 2021, Art. no. 103029.
- [13] W. Xu, Z. Zhang, X. You, and C. Zhang, "Reconfigurable and low-complexity accelerator for convolutional and generative networks over finite fields," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 39, no. 12, pp. 4894–4907, Dec. 2020.
- [14] Z. Baozhou, N. Ahmed, J. Peltenburg, K. Bertels, and Z. Al-Ars, "Diminished-1 fermat number transform for integer convolutional neural networks," in *Proc. IEEE 4th Int. Conf. Big Data Analytics (ICBDA)*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 47–52.
- [15] T. Toivonen and J. Heikkilä, "Video filtering with Fermat number theoretic transforms using residue number system," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 16, no. 1, pp. 92–101, Jan. 2006.
- [16] X. Chen, B. Yang, S. Yin, S. Wei, and L. Liu, "CFNTT: Scalable radix-2/4 NTT multiplication architecture with an efficient conflict-free memory mapping scheme," *IACR Trans. Cryptogr. Hardware Embedded Syst.*, vol. 2022, no. 1, pp. 94–126, Nov. 2021.
- [17] J. Mu et al., "Scalable and conflict-free NTT hardware accelerator design: Methodology, proof, and implementation," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 5, pp. 1504–1517, May 2023.
- [18] Y. Zhang et al., "Ultra high-speed polynomial multiplications for lattice-based cryptography on FPGAs," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 10, no. 4, pp. 1993–2005, Oct.–Dec. 2022.
- [19] Z. Cheng, B. Zhang, and M. Pedram, "A high-performance, conflict-free memory-access architecture for modular polynomial multiplication," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 2, pp. 492–505, Feb. 2024.
- [20] Y. Zhao, X. Liu, Y. Hu, and H. Xiao, "Design of an efficient NTT/INTT architecture with low-complex memory mapping scheme," *IEEE Trans. Circuits Syst., II, Exp. Briefs*, vol. 71, no. 1, pp. 400–404, Jan. 2024.
- [21] P. Duong-Ngoc and H. Lee, "Configurable mixed-radix number theoretic transform architecture for lattice-based cryptography," *IEEE Access*, vol. 10, pp. 12732–12741, 2022.
- [22] G. Li, D. Chen, G. Mao, W. Dai, A. I. Sanka, and R. C. Cheung, "Algorithm-hardware co-design of split-radix discrete Galois transform for KyberKEM," *IEEE Trans. Emerg. Topics Comput.*, vol. 11, no. 4, pp. 824–838, Oct.–Dec. 2023.
- [23] W. Guo and S. Li, "Split-radix based compact hardware architecture for crystals-kyber," *IEEE Trans. Comput.*, vol. 73, no. 1, pp. 97–108, Jan. 2024.
- [24] N. Zhang, B. Yang, C. Chen, S. Yin, S. Wei, and L. Liu, "Highly efficient architecture of NewHope-NIST on FPGA using low-complexity NTT/INTT," *IACR Trans. Cryptographic Hardware Embedded Syst.*, vol. 2020, no. 2, pp. 49–72, Mar. 2020.
- [25] B. Li, Y. Yan, Y. Wei, and H. Han, "Scalable and parallel optimization of the number theoretic transform based on FPGA," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 32, no. 2, pp. 291–304, Feb. 2024.
- [26] A. C. Mert, E. Karabulut, E. Öztürk, E. Savaş, and A. Aysu, "An extensive study of flexible design methods for the number theoretic transform," *IEEE Trans. Comput.*, vol. 71, no. 11, pp. 2829–2843, Nov. 2022.
- [27] S. S. Roy, F. Vercauteren, N. Mentens, D. D. Chen, and I. Verbauwhede, "Compact ring-lwe cryptoprocessor," in *Proc. Int. Workshop Cryptogr. Hardware Embedded Syst.*, Berlin, Germany: Springer, 2014, pp. 371–391.
- [28] T. Pöppelmann, T. Oder, and T. Güneysu, "High-performance ideal lattice-based cryptography on 8-bit atmega microcontrollers," in *Proc. Int. Conf. Cryptol. Inf. Secur. Latin Amer.*, Cham, Switzerland: Springer, 2015, pp. 346–365.
- [29] R. Agarwal and C. Burrus, "Fast convolution using Fermat number transforms with applications to digital filtering," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 22, no. 2, pp. 87–97, Apr. 1974.
- [30] H.-F. Lo, M.-D. Shieh, and C.-M. Wu, "Design of an efficient FFT processor for DAB system," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS) (Cat. No. 01CH37196)*, vol. 4, Piscataway, NJ, USA: IEEE Press, 2001, pp. 654–657.
- [31] D. D. Chen et al., "High-speed polynomial multiplication architecture for ring-LWE and SHE cryptosystems," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 62, no. 1, pp. 157–166, Jan. 2015.
- [32] U. Banerjee, T. S. Ukyab, and A. P. Chandrakasan, "Sapphire: A configurable crypto-processor for post-quantum lattice-based protocols," 2019, *arXiv:1910.07557*.
- [33] L. Johnson, "Conflict free memory addressing for dedicated FFT hardware," *IEEE Trans. Circuits Syst. II, Analog Digit. Signal Process.*, vol. 39, no. 5, pp. 312–316, May 1992.
- [34] X. Chen, B. Yang, Y. Lu, S. Yin, S. Wei, and L. Liu, "Efficient access scheme for multi-bank based NTT architecture through conflict graph," in *Proc. 59th ACM/IEEE Des. Automat. Conf.*, 2022, pp. 91–96.



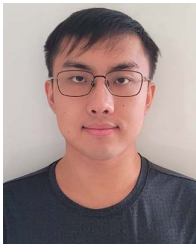
Yile Xing (Student Member, IEEE) received the B.Eng. degree from the School of Electronics and Communication Engineering, Sun Yat-sen University, China, in 2021. She is currently working toward the Ph.D. degree with the Department of Electrical Engineering, City University of Hong Kong. Her research interests include digital signal processing algorithm design and reconfigurable computing with FPGA.



Guangyan Li received the B.Eng. degree from the Department of Electrical Engineering, City University of Hong Kong, in 2020. He is currently working toward the Ph.D. degree with the Department of Electrical Engineering, City University of Hong Kong. His research interests include reconfigurable computing with FPGA, and postquantum cryptography algorithm design.



Zewen Ye (Student Member, IEEE) received the bachelor's degree in microelectronics science and engineering from Zhejiang University, in 2020. He is currently working toward the joint Ph.D. degree with Zhejiang University and the City University of Hong Kong, advised by Prof. Kejie Huang and Prof. Ray C. C. Cheung. His research interests include postquantum cryptography, hardware design, and RISC-V.



Ryan W. L. Luk (Member, IEEE) received the B.Eng. degree in electronic engineering from The Hong Kong University of Science and Technology, in 2021. Since 2022, he has been working toward the M.Sc. degree in electrical engineering with the City University of Hong Kong. Currently, he is a Research Assistant supervised by Dr. Ray C. C. Cheung with the Department of Electrical Engineering, City University of Hong Kong. His research interests include computer arithmetic and modulo-multiplier design.



Donglong Chen (Member, IEEE) received the Ph.D. degree from the Department of Electronic Engineering, City University of Hong Kong, in 2015. He was a Visiting Research Scholar of COSIC, KU Leuven, Belgium, in 2013. After completing his Ph.D. degree study, he spent four years with the industry including Huawei Technology Company Ltd., and Tencent Technology Company Ltd. Currently, he is an Associate Professor with the Faculty of Science and Technology, Beijing Normal-Hong Kong Baptist University Zhuhai, China. His research interests include cryptographic engineering, software/hardware co-design for AI algorithms, and privacy computing.



Hong Yan (Life Fellow, IEEE) received the Ph.D. degree from Yale University. He was a Professor of imaging science with the University of Sydney, and currently a Wong Chun Hong Professor of data engineering and a Chair Professor of computer engineering with the City University of Hong Kong. His research interests include image processing and computer vision, machine learning, and computational biology and medicine. He has over 600 journal and conference publications in these areas. He received the 2016 Norbert Wiener Award from

the IEEE SMC Society for contributions to image and biomolecular pattern recognition techniques. He is an IAPR Fellow, a Foreign Member of the European Academy of Sciences and Arts, and a fellow of the US National Academy of Inventors.



Ray C. C. Cheung (Senior Member, IEEE) received the B.Eng.(Hons.) and M.Phil. degrees in computer engineering and computer science & engineering from The Chinese University of Hong Kong (CUHK), in 1999 and 2001, respectively, and the DIC and Ph.D. degrees in computing from Imperial College London (IC), in 2007. Currently, he is the Associate Provost (Digital Learning) with the CityUHK, a Professor with the Department of Electrical Engineering and Department of Computer Science at CityUHK, and the CityUHK-EE Xilinx

Lab. He is the current Section Chairman of IEEE HK Section, the Former Chairman of the IEEE Hong Kong Section CAS/COM Chapter, and an Executive Committee Member of the IEEE Hong Kong Section Computer Chapter. His research interests include cryptographic hardware designs and design exploration of system-on-chip (SoC) designs, AIoT designs, and embedded system designs.